

FPGA implementation of feature detection and matching using ORB

Nadir Mohamed Belmessaoud^{a,*}, Youcef Bentoutou^b, Miloud Chikr El-Mezouar^a

^a RCAM Laboratory, Department of Electronics, Djillali Liabes university, Sidi Bel Abbas, 22084, Sidi Bel Abbas, Algeria

^b Satellite Development Center, Algerian Space Agency, Bir El Djir, 31130, Oran, Algeria

ARTICLE INFO

Keywords:

FPGA
Feature detection
Feature matching
Oriented FAST and Rotated BRIEF

ABSTRACT

Oriented FAST and Rotated BRIEF (ORB) is becoming the method of choice in real-time feature matching applications by offering performances comparable to other state-of-the-art descriptors at a fraction of the runtime and memory requirements. These characteristics have prompted multiple researchers to propose FPGA-based accelerators that allow for a significant increase in throughput compared to general-purpose processors. This increase is, however, achieved at the expense of the descriptor's robustness as several approximations are necessary to map the algorithm efficiently into hardware which negatively affects its invariance. In this paper, we introduce an FPGA architecture capable of extracting ORB descriptors at high throughput with minimal impact to their invariance. Rotation of sample point coordinates is bypassed entirely, and methods for the accurate computation of the moments and descriptor generation in a single clock cycle are described. Moreover, a feature matching algorithm based on the cross-check criterion is proposed and implemented in hardware. The architecture achieved a throughput of 94 MPix/s using a 100 MHz clock while producing an almost identical performance compared to the reference implementation.

1. Introduction

Computer vision is at the forefront of countless technological endeavors. From autonomous vehicles to augmented reality and robot navigation, a broad spectrum of modern applications relies on machine vision algorithms to operate. And with feature matching being an integral part of a significant subset of these algorithms, the need arises for computationally efficient and robust feature detectors. The relatively recent emergence of binary descriptors came as a response to this need since the high complexity of well-established descriptors such as Scale-Invariant Feature Transform (SIFT) [1] and Speeded Up Robust Feature (SURF) [2] is prohibitive in most real-time applications.

The advent of local binary descriptors originated with the introduction of Binary Robust Independent Elementary Features (BRIEF) [3], which generates the descriptor bits by carrying out intensity comparisons of pixels within a local patch centered at the keypoint. BRIEF is able to generate highly discriminative descriptors while requiring minimal computations and memory footprint. But its intrinsic deficiencies, namely its sensitivity to in-plane rotation and scale changes, make it non-viable as an alternative to SIFT or SURF. ORB, as introduced in [4], improves upon BRIEF by achieving invariance to scale and rotation using a scale pyramid representation of the input image and an orientation compensation mechanism. Additionally, it employs a greedy search algorithm to locate a set of test pairs whose high variances

are preserved when the sampling pattern is steered according to the orientation of the keypoint. These improvements culminated in ORB showing similar performances to SIFT [5] and outperforming its binary counterparts in several test cases [6,7].

Although hardware implementations of SIFT and SURF that are capable of extracting features at a very high throughput have been proposed [8,9], a more efficient accelerator can be conceived by leveraging the binary nature of ORB and the use of Hamming distance in feature matching. Over the last decade, several Field-Programmable Gate Array (FPGA) architectures have been proposed seeking to leverage the inherent parallelism and pipelined nature of these devices to create stream processing focused designs. However, when comparing these architectures with the reference implementation of ORB in OpenCV [10], some discrepancies can be observed. Notably: the use of a square window in the calculation of the moments instead of a circular one, approximating atan2, sin, and cos functions with look-up tables, and conducting the intensity tests on the non-smoothed input image.

The aforementioned dissimilarities can severely impact the descriptor's invariance to rotation and noise. This loss of invariance might be tolerable when the features being matched are from successive frames of a camera feed where the relative distortion between any two consecutive frames is minimal. But with ORB being used in applications where

* Corresponding author.

E-mail addresses: mohamed.belmessaoud@univ-sba.dz (N.M. Belmessaoud), bentoutou@asal.dz (Y. Bentoutou), miloud.chikrelmezouar@univ-sba.dz (M. Chikr El-Mezouar).

<https://doi.org/10.1016/j.micpro.2022.104666>

Received 20 December 2021; Received in revised form 28 June 2022; Accepted 25 August 2022

Available online 30 August 2022

0141-9331/© 2022 Elsevier B.V. All rights reserved.

the images have considerably more pronounced distortions, e.g., image registration and image mosaicing [11,12], the loss of invariance can be problematic. In this paper, we propose an FPGA architecture for the extraction and matching of ORB features for stream processing applications without sacrificing the robustness of the algorithm. The system achieved a performance similar to the reference implementation in OpenCV with a frame rate of 163 fps when tested on images having a resolution of 850×680 pixels.

1.1. Related works

To our knowledge, [13] was the first attempt to map ORB onto an FPGA which integrated the Features from Accelerated Segment Test (FAST) [14] detector with the rBRIEF descriptor. The paper reported a latency of 9 ms for 640×480 images using a 100 MHz clock. [15] presents a hardware ORB feature extractor targeted at Simultaneous Localization And Mapping (SLAM) applications. A word reduction scheme for the patch moments is introduced to minimize resource utilization. Details concerning how the moments were computed and how the descriptor was extracted were omitted from both papers.

The accelerator proposed in [16] modifies the ORB algorithm by replacing the Harris metric computation by a Sum of the Absolute Differences (SAD) and the centroid-based orientation measurement by the method used in [17]. It utilizes a data reuse scheme in which two keypoints can share a section of the same patch if they are both located within it. [18] extended the work done in [16] by incorporating the design in a block-wise serial-hybrid architecture where three scales of a given octave are processed in parallel and multiple octaves are processed sequentially. The orientation of the keypoint is attributed using an approximation of the intensity centroid which is implemented using a multibank memory scheme. Additionally, a 3×3 non-maximum suppression with heap-sorting is used to balance the keypoint distribution over the image. The architecture uses the same smoothed image data for the centroid computation and the descriptor generation.

The architecture proposed in [19] constructs a 4-level image pyramid where each level consists of a feature detection component, a down sample component that outputs an image downsampled by a factor of 1.25 to a higher level, and a smoothing component. Each feature detection component is based on a FAST keypoint detector that utilizes SAD as the corner score function. The output of the smoothing components are stored in an external memory to save on-chip resources. The four levels of the pyramid run in parallel and the keypoints detected in each level are stored in a keypoint buffer. The computation of the descriptor and the orientation for each feature point is done sequentially where a patch loader reads the smoothed local patch around the keypoint from the external memory, which obviously leads high external memory bandwidth requirements. To remedy this, a data reuse scheme similar to [16] is used, along with storing the smoothed images in every other scale. The same filtered image patch is used in the computation of the orientation and the descriptor and the descriptor is computed in 256 clock cycles. [20] proposes a new set of test pairs seeking to leverage a memory allocation scheme in which 8 pixels are retrieved every clock cycle, resulting in 17 times speed-up over the traditional rBRIEF vector. The implementation assumes that the filtered image is stored in a dual port RAM and did not include an orientation compensation mechanism.

[21] introduces an ORB implementation targeted at visual SLAM applications. Similar to [20], this work aims to make the computation of the ORB descriptor more hardware-friendly by modifying the original sampling pattern. The new pattern, called the rotationally symmetric BRIEF (RS-BRIEF), allows the operation of rotating test locations to be replaced by a simple shift of the generated descriptor. Compared with Intel i7 CPU and the ARM Cortex A-9 process, the architecture achieved 3.6 $\times$ and 32 $\times$ speedup in feature extraction, 4.9 $\times$ and 61.6 $\times$ speedup in feature matching. The same smoothed image patch was used for both descriptor generation and orientation computation and details concerning the hardware implementation of the latter were not

elaborated. [22] presents a concept design for an FPGA-based ORB accelerator that consists of a combination of FAST and rBRIEF. An image pyramid is constructed from the input image where each level is downsampled from the previous one by a factor of $\frac{4}{5}$ using bilinear interpolation and heap sorting is used to select N feature points with the highest SAD score. The design is, however, evaluated using Python and a hardware implementation was not developed.

The previously mentioned architectures require constant access to the external memory during the detection and descriptor generation stages making them unfit for streaming applications. The more stream oriented works include [23], which combined the Harris detector [24] with the rBRIEF descriptor. The same 37×37 -pixel patch was used for the moments computation and descriptor generation. One intensity comparison was conducted per clock cycle, leading to the full descriptor requiring 256 clocks during which the module is unable to accept input data, so any keypoint detected during that time frame is dropped. To minimize the number of dropped keypoints, the module is instantiated four times, which reduces the runtime to 64 clock cycles. [25] extended the work done in [23] by adding scale invariance by means of multi-level image pyramids. A 9 level pyramid with 16 instantiations of the ORB module spread across them was deemed acceptable according to their experimentation. [26] introduced a deep-pipelined architecture for extracting rBRIEF descriptors which can be paired with any standard feature detector. A moving summation strategy was elaborated for the calculation of the moments and a time-multiplexing scheme is used to rotate the test coordinates based on the assumption that the feature detection stage included a 7×7 Non-Maximum Suppression (NMS) operation. Multiplier-less circuitry is used throughout the design in order to reduce the hardware utilization and power consumption. In addition to that, the paper investigated the impact of the choice of several parameters on the accuracy of the algorithm, including the number of bits allocated for the moments and for the sine and cosine of the angle of orientation. The approach proposed in [27] consists in dividing the image into non-overlapping regions called buckets and preserving only the best keypoints in each bucket. A decoupled workflow is adopted in which intermediate results are stored in a keypoint buffer instead of integrating all of the components in a single pipeline. Moreover, the rotation of the sampling pattern is avoided, and the image patch is rotated instead. However, the hardware implementation was not sufficiently detailed.

1.2. Contributions

The primary goal of this work is the design of a high throughput FPGA-based implementation of ORB without sacrificing the robustness of the algorithm, namely its invariance to rotation and noise. Based on the literature review above, we identify three main areas that cause the accuracy of a hardware adaptation of the algorithm to be subpar compared to its software counterpart. The first issue relates to the computation of the patch moments. [23] reported that one of the major causes for the difference between the FPGA and the software implementations is due to the differently-shaped windowing function used in the moments calculation. The second area concerns the rotation of the sampling pattern, where using a limited number of bits to represent the sine and cosine of the angle of orientation can severely impact the accuracy, as investigated in [26]. Finally, computing the descriptor one bit at a time can cause several keypoints to be dropped, leading to some keypoints being present in one image but absent in a subsequent one.

The contributions of this paper are therefore the following:

- A hardware structure is proposed that allows a stream-oriented FPGA implementation of ORB to efficiently produce the same values for the moments as the software implementation without storing the image patch.

- Rotation of the sampling pattern is bypassed entirely, therefore eliminating the errors introduced by approximating the sine and cosine of the angle of orientation with limited precision.
- All 256 bits of the descriptor are computed in a single clock cycle without altering the original sampling pattern of ORB.
- Additionally, a feature matcher based on the cross-check criterion is implemented in hardware.

The organization of the paper is as follows: Section II provides a background on the ORB algorithm while Section III details the proposed architecture. The results of our experimentation are presented and discussed in Section IV. Finally, the related conclusions are given in Section V.

2. The ORB algorithm

ORB constructs a multi-scale representation of the input image where each level is downsampled from the previous one by some factor, it then conducts feature extraction which comprises a keypoint detection and descriptor generation components. The former utilizes a combination of FAST and Harris named oriented FAST (oFAST), while the latter is a modification on BRIEF labeled the rotation-aware BRIEF (rBRIEF).

2.1. oFAST

The FAST corner detector is used as the initial stage in oFAST. It selects candidate points based on intensity tests between a center pixel and the pixels on the Bresenham circle with a radius of 3 centered on it. The center pixel is considered a corner if there exist n contiguous pixels on the circle that are all brighter than it plus a threshold, or all darker minus a threshold, with n being chosen as 9 in oFAST. Fig. 1 shows a keypoint detected by FAST using a threshold of 150. The corner is outlined in blue, while the Bresenham circle consists of the pixels outlined in green and red. The green pixels represent the 9 contiguous points in the circle whose intensity value is less than that of the keypoint minus the threshold of 150. The intensity of the red pixels is neither less than that of the keypoint minus the threshold, nor greater than the intensity of the central pixel plus the threshold.

FAST provides a boolean response indicating whether a given pixel is a corner or not. But it is often desirable to obtain a quantitative value indicating the strength of the detected keypoint, which can subsequently be used to either sort the corners or in non-maximum suppression to avoid having multiple keypoints adjacent to one another. To this end, oFAST uses the Harris metric as a corner score, which is computed for every keypoint detected by FAST. The keypoints are then sorted based on their score and the N features with the highest score are chosen.

To achieve invariance to rotation, oFAST computes the orientation of each feature point by means of the intensity centroid, which can be obtained by calculating the moments within a patch centered at the keypoint. The $(p + q)^{\text{th}}$ order moment is defined as:

$$m_{pq} = \sum_{x,y} x^p y^q I(x, y) \quad (1)$$

where $I(x, y)$ is the pixel intensity of the image patch at coordinates x and y .

The orientation of the corner is then obtained by:

$$\theta = \text{atan2}(m_{01}, m_{10}) \quad (2)$$

[4] emphasizes the requirement of computing the moments with x and y remaining within a circular region of radius r in order to improve the rotation invariance of the measure.

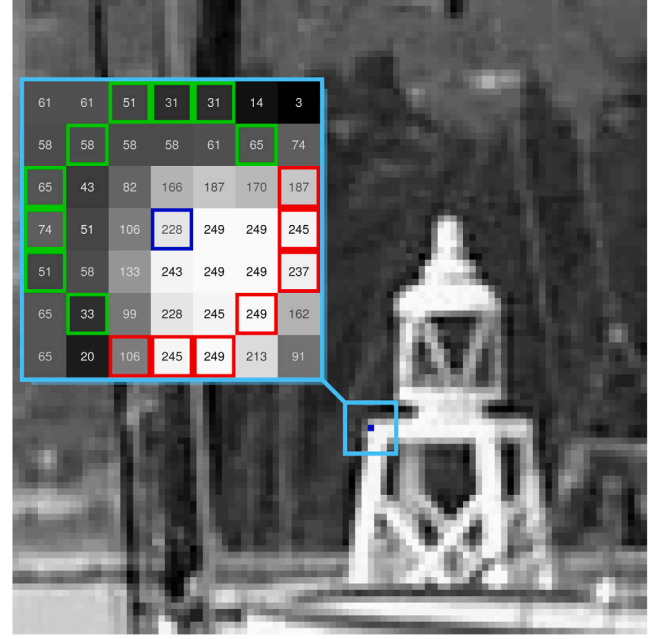


Fig. 1. Example of a corner detected by FAST with a threshold of 150. The number on each pixel represents its intensity.

2.2. rBRIEF

The second stage of ORB is the descriptor extraction, wherein a binary vector is computed that represents the local environment around a keypoint. To this end, ORB employs rBRIEF, which consists of a sampling pattern of 512 sample points arranged as 256 point pairs. rBRIEF achieves invariance to rotation by steering the sampling pattern according to the orientation of the keypoint whose descriptor is being computed, which is done by rotating the coordinates of each sample point using the following transformation:

$$\begin{bmatrix} x_r \\ y_r \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (3)$$

Intensity tests are then carried out between the two rotated sample points in each of the 256 pairs. Given a smoothed image patch p centered at a detected keypoint, the intensity tests are defined in [4] as:

$$\tau(p; x, y) := \begin{cases} 1 & : p(x) < p(y), \\ 0 & : p(x) \geq p(y) \end{cases} \quad (4)$$

where $p(x)$ represents the intensity of the patch p at point x . The output of all intensity tests result in the 256-bit rBRIEF descriptor.

3. The proposed architecture

The proposed architecture is given in Fig. 2. Fig. 2(a) depicts an overview of components used within the target Zynq System on Chip (SoC) device. The Processing System (PS) reads the images from the DDR3 memory and sends them through the AXI4 High Performance (HP) port to the ORB accelerator in the Programmable Logic (PL), which reads them using the AXI4-Stream interface. The results of the processing, which consist of the coordinates of the keypoints from the two images and the matches are read by the PS using an AXI4 General Purpose (GP) port from the AXI4-lite slave interface on the PL. The accelerator signals the end of processing to the PS using two interrupt lines, the first indicates the end of extracting the keypoints and their descriptors from an image and the second is asserted when the feature matching procedure is concluded.

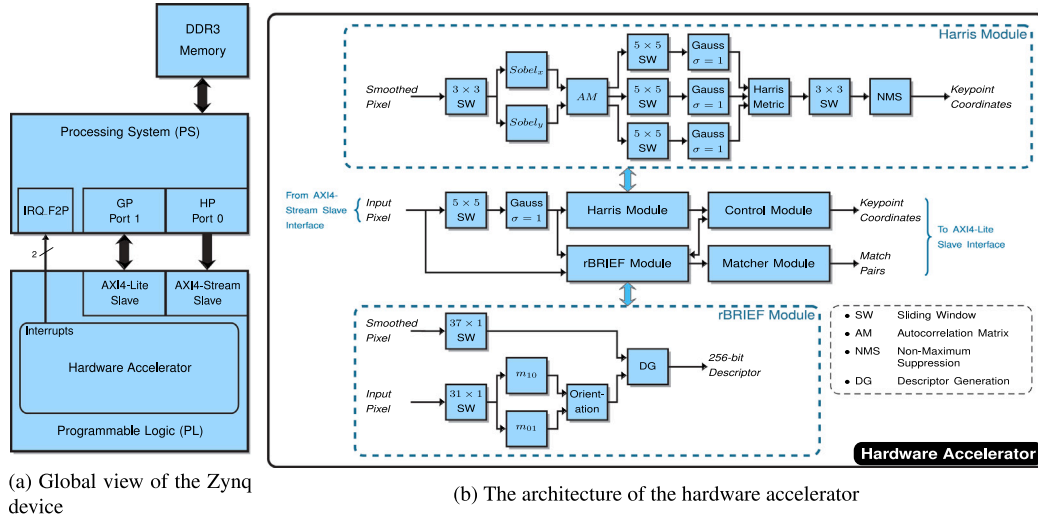


Fig. 2. Overview of the proposed architecture.

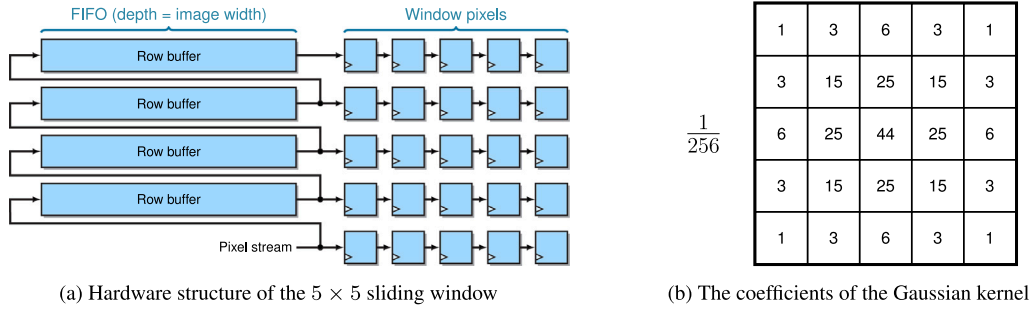


Fig. 3. The two components of the Gaussian filter.

Fig. 2(b) provides the hardware architecture of the ORB accelerator. The input pixel stream branches into two paths. The first path passes through a Gaussian filter before being fed to the Harris module and the 37×1 sliding window in the rBRIEF module. The second path routes the input stream directly to the 31×1 sliding window which outputs the patch for the moments computation.

As stated above, ORB uses FAST to select a set of candidate keypoints and then computes the Harris response for this set. This is done to avoid calculating the Harris response for the entire image which can be demanding for a software implementation. In contrast, FAST is not incorporated in the proposed FPGA-based accelerator since the Harris response is computed for the entire image. Additionally, the current implementation extracts features from the input image without recurring to multi-scale representations. Constructing an image pyramid with the current implementation running in each level similar to [25] can be used to cope with the scale change problem.

The three main modules in the proposed architecture are Harris, rBRIEF and the feature matcher, with the control module serving mainly as a coordinator.

3.1. Harris module

The architecture of the Harris feature detection module is depicted in the upper section of Fig. 2(b). Unlike previous implementations of Harris, e.g. [23,28,29], the input to this module is the filtered image data instead of raw pixels since it has been shown in [30] that pre-filtering the image improves the repeatability of the detector. Filtering throughout the design is achieved using 5×5 Gaussian kernels whose coefficients are chosen according to the fixed-point approximation proposed in [31]. Fig. 3 depicts the two elements involved in the filtering process, the sliding window and the Gaussian kernel.

The smoothed pixel stream is passed to two Sobel components through a 3×3 sliding window where the horizontal gradient G_x and the vertical gradient G_y are computed. These two gradients are then used to obtain the elements of the autocorrelation matrix M , which is defined as:

$$M = \begin{bmatrix} G_x^2 & G_x \cdot G_y \\ G_x \cdot G_y & G_y^2 \end{bmatrix} \quad (5)$$

Three parallel sliding windows followed by Gaussian kernels are used to filter the elements of M resulting in the matrix M' . The Harris response is then obtained by:

$$R = \det(M') - k \cdot \text{trace}^2(M'), \quad (6)$$

where k is an empirically determined parameter, whose value is typically between 0.04 or 0.06 [30].

The final component of the feature detection module is non-maximum suppression, where a candidate pixel is considered a key-point if three main criteria are met: First, the pixel's response is strictly greater than its 8 connected neighbors. Second, the response is above a given threshold. Finally, the distance between it and the previously detected keypoint is above a certain threshold. The metric used in the last criterion is the efficient Chebyshev distance which, given two pixels pxl_0 and pxl_1 with coordinates (x_0, y_0) and (x_1, y_1) respectively, can be computed by:

$$D = \max(|x_1 - x_0|, |y_1 - y_0|) \quad (7)$$

The purpose of imposing a distance threshold between two consecutive keypoints is twofold. First, it allows for a better distribution of the features over the entire image, and second, it ensures the minimum distance required by the descriptor generation component in the rBRIEF module.

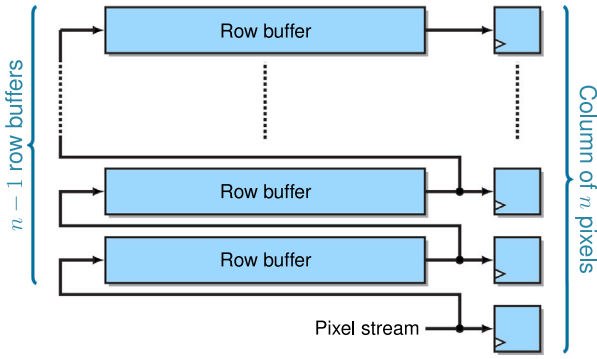


Fig. 4. The hardware structure of a generic $n \times 1$ sliding window.

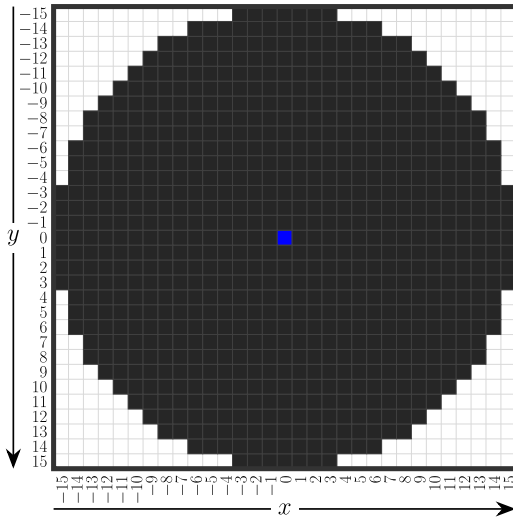


Fig. 5. The circular window used in the computation of the moments.

The output of the Harris module is a coordinate pair of the detected keypoints which are then stored in a Block Random Access Memory (BRAM) in the control module. The BRAMs consist of a set of primitives within the PL of the Zynq SoC that can implement dual-port or single port RAMs with adjustable widths and depths.

3.2. rBRIEF module

The rBRIEF module is composed of three main units: **moments computation, orientation estimation and descriptor generation**. The 37×1 sliding window outputs a column of 37 filtered pixels to the descriptor generator while the 31×1 window outputs a column of 31 unfiltered pixels to the moments units. Fig. 4 illustrates the hardware structure of a generic $n \times 1$ window.

3.2.1. The moments unit

Given a 31×31 image patch, the objective of this unit is to produce the moments m_{10} and m_{01} based only on the pixels within a circular region of radius $r = 15$. Fig. 5 shows the window used in the reference implementation of ORB, the blue pixel in the center represents the key-point whose moments are to be computed and also serves as the origin of the coordinate system in the moments computation, while the dark pixels correspond to the locations that are used in the computation. The motive behind using this shape is to improve the rotational invariance of the algorithm, as approximately the same pixels will be used in the calculations regardless of image rotation.

The straightforward approach used in [15] consists in storing the entire 31×31 -pixel patch in a 2D register file, which will inevitably

lead to high resource utilization. The use of a circular window is avoided in [23,25,26], resulting in more efficient but less accurate implementations.

The approach adopted in this work is as follows, starting by the moment m_{10} , Eq. (1) can be rewritten as:

$$m_{10} = \sum_{x=-r}^r x \sum_{y=-v(x)}^{v(x)} I(x, y), \quad (8)$$

where

$$v(x) = \left\lfloor \frac{c(x)}{2} \right\rfloor \quad (9)$$

$c(x)$ represents the number of pixels to be used in the computation in column x .

Defining $\sum_{y=-v(x)}^{v(x)} I(x, y)$ as $S_{c(x),x}$, where $S_{c(x),x}$ is the sum of the $c(x)$ middle pixels in column x , we obtain

$$m_{10} = \sum_{x=1}^r x(S_{c(x),x} - S_{c(x),-x}) \quad (10)$$

As the window shown in Fig. 5 passes over the image, different values of $S_{c(x)}$ need to be calculated for the same column depending on its position within the window. Our implementation consists in calculating all the summations that are needed for every column that has been output by the 31×1 sliding window. Particularly, $S_7, S_{13}, S_{17}, S_{19}, S_{21}, S_{23}, S_{25}, S_{27}, S_{29}$ and S_{31} . Each summation $S_{c(x)}$ is subsequently stored in a shift register $SR_{S_{c(x)}}$ where elements in locations x and $-x$ can be retrieved. After having retrieved the elements, obtaining m_{10} is relatively straightforward.

Instead of using a separate adder tree for each summation, we devise the adder depicted in Fig. 6(a), which receives a column of 31 pixels as input and produces all of the aforementioned summations simultaneously.

The computation of m_{01} is achieved in a similar manner. By rewriting Eq. (1) as:

$$\begin{aligned} m_{01} &= \sum_{x=-r}^r \sum_{y=-v(x)}^{v(x)} y \cdot I(x, y) \\ &= \sum_{x=-r}^r \sum_{y=1}^{v(x)} y(I(x, y) - I(x, -y)) \end{aligned} \quad (11)$$

and defining $\sum_{y=1}^{v(x)} y(I(x, y) - I(x, -y))$ as $W_{v(x),x}$, it is possible to compute all of the $W_{v(x)}$ values for each column that has been output by the 31×1 sliding window, i.e., $W_3, W_6, W_8, W_9, W_{10}, W_{11}, W_{12}, W_{13}, W_{14}$, and W_{15} . Each $W_{v(x)}$ value is thereafter loaded into a corresponding shift register $SR_{W_{v(x)}}$ and m_{01} is obtained by fetching the appropriate elements from the registers and summing them.

Similar to how the $S_{c(x)}$ values were calculated, computing the $W_{v(x)}$ values for a given column is done using a single pipelined adder as shown in Fig. 6(b).

Fig. 7 illustrates the hardware architecture of the m_{10} and m_{01} units. The number on each location in the shift registers corresponds to its x coordinate within the 31×31 pixel patch, and with the locations involved in the computation from each register shown in full opacity.

3.2.2. The orientation unit

After having obtained the moments, the next step is to impute an orientation for the keypoint. Similar to [23], we employ 64 angle quantisations leading to a maximum error of 2.81° . The steps involved in the orientation estimation consist of determining the quadrant of the angle based on the signs of the two moments. The latter are then rotated so that the angle falls in the $[0; \frac{\pi}{2})$ quadrant. Among the 16 angles in this quadrant, the index j of the angle θ_j that best satisfies the equation

$$m_{01} = \tan(\theta_j) \times m_{10} \quad (12)$$

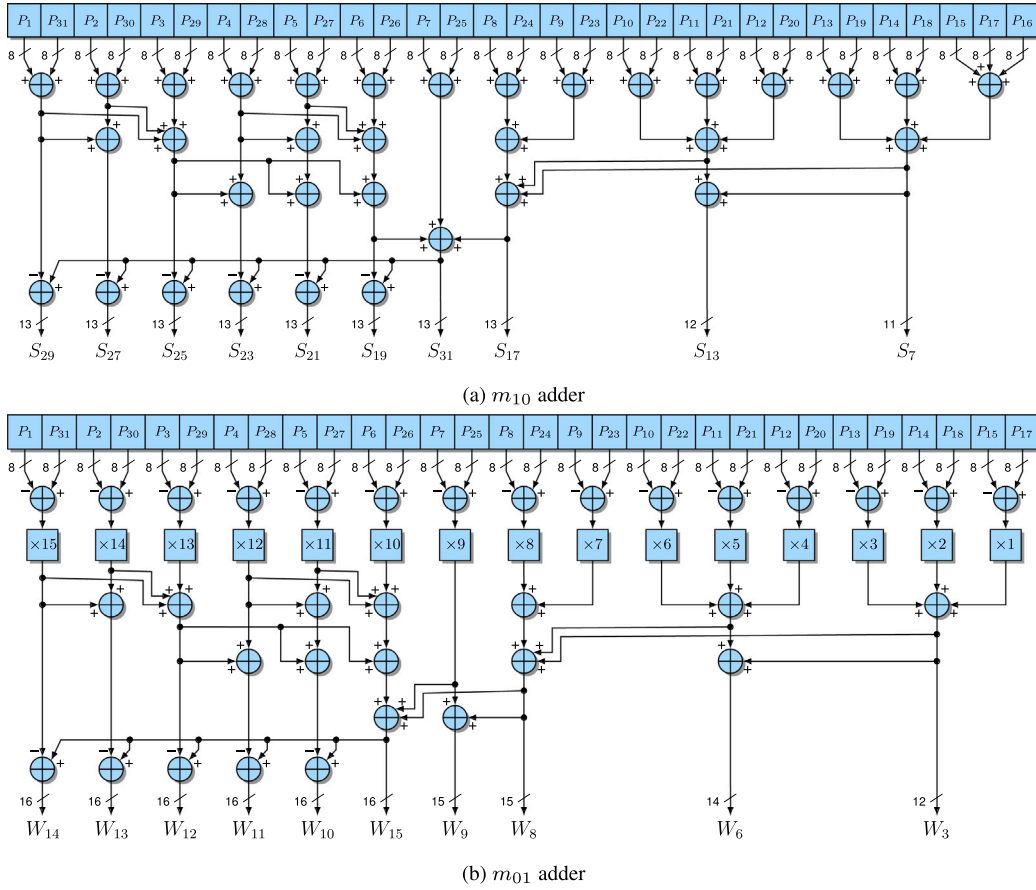


Fig. 6. The adders used to obtain the $S_{c(x)}$ and $W_{c(x)}$ values for each column. Pipelining registers have been omitted for clarity.

is chosen. Finally, a constant whose value is either 0, 16, 32 or 48 depending on whether the angle was initially determined to be in the first, second, third or fourth quadrant respectively is added to the index j to obtain the index $angle_idx$ which corresponds to the true orientation of the keypoint.

3.2.3. The descriptor generation unit

The objective of this unit is to construct the descriptor of a given keypoint with the added constraint of computing the 256 bits in a single clock cycle. Previous implementations resorted to generating the descriptor one bit at a time mainly due to two obstacles. The first consists in the need to rotate the 512 sample coordinates using Eq. (3), which would result in 1024 additions and 2048 multiplications.

[26] introduced a method for eliminating the multiplications using Multiple Constant Multiplication (MCM) blocks. The number of additions can be reduced by observing that even though the sampling pattern of ORB consists of 512 points, not all of them are unique, and a single sample point can be used in multiple binary tests. By removing duplicates, the number of samples can be reduced from 512 to 375 unique points, requiring 750 additions instead of 1024. Even with these optimizations, the resources needed to implement the rotation of sample points would be substantial.

The second obstacle is caused by storing the 37×37 patch used for the sampling process in dual-port BRAMs, which allows the retrieval of only two pixels per clock cycle.

Given a column n output by the 37×1 sliding window, where n is the position of this column within the 37×37 sampling patch, the value of sample point i with rotated coordinates (x_{r_i}, y_{r_i}) can be assigned to it using the approach depicted in Fig. 8. When the column index is equal to x_{r_i} , the register storing the value of the sample point is enabled and y_{r_i} is used to select the pixel of interest.

While this method eliminates the second obstacle, it requires a large multiplexer to be associated with each of the 375 unique sampling points, and the need to obtain the rotated coordinates is still present. The following outlines the steps that were undertaken to improve on this approach.

The first step is to reduce the size of the multiplexer. As a sample point rotates, its ordinate values will be in the range $[-y_{max}, y_{max}]$, using a $37 : 1$ multiplexer is necessary only when $y_{max} = 18$, otherwise a smaller multiplexer with a size $l = 2 \cdot y_{max} + 1$ can be used to select one of the l middle elements of the incoming 37-pixel column.

Subsequently, a brute-force search was conducted to find a group of sample points from the set of 375 points that, when rotated by any of the 64 angles that can be attributed to a keypoint as an orientation, would never fall in the same column. Therefore, a single multiplexer can be shared by all of these points. This group of sample points will henceforth be referred to as a *batch*. The size L of the shared multiplexer corresponds to the size of the largest multiplexer that is associated with a sample point in the current batch.

The information pertaining to when the register storing a sample point is enabled and the corresponding multiplexer select value are kept in a BRAM-based look-up table whose address is a concatenation of the column index n and the angle index $angle_idx$ of the keypoint whose descriptor is being extracted. Fig. 9 illustrates the resulting design from these improvements.

The improved design significantly reduces the hardware utilization over the straightforward approach and eliminates the need to obtain the rotated coordinates, since this information has been pre-calculated and stored in the BRAM.

A further optimization consists in grouping batches into pairs as depicted in Fig. 10. The selection of whether two batches can be grouped together is made by rotating the sample points in them by

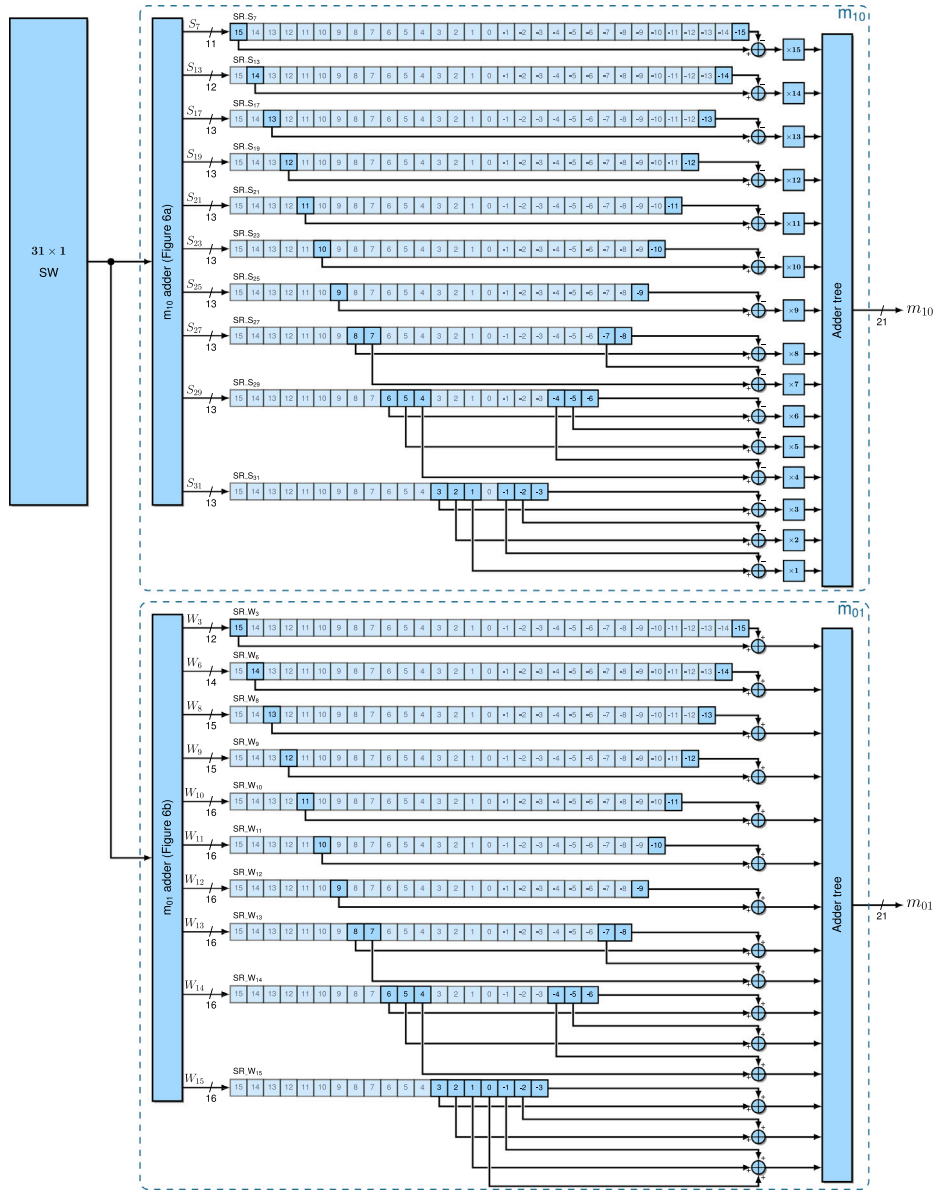


Fig. 7. Hardware architecture for implementing the moments computation units.

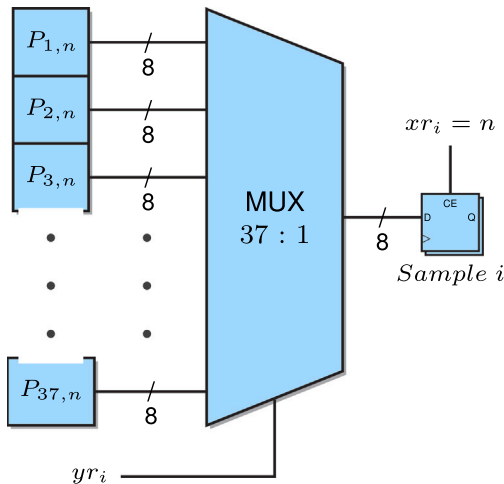


Fig. 8. Simple approach for assigning a value to a sampling point.

all 64 angles, if the number of collisions, i.e., two points from the two batches falling in the same column, is less than a predefined threshold K , then these two batches can be grouped and a small $K : 1$ multiplexer can be used for the second batch. K was chosen as 16 in our implementation, since 16 : 1 multiplexers can be efficiently implemented using a single slice in the target Zynq FPGA. If there is no collision in column n , then a given pixel is either routed to the first batch using MUX_1, or to the second batch through MUX_1 then MUX_2. If a collision occurs, then the pixel needed for the first batch is selected by MUX_1, and the pixel needed for the second batch is connected directly to an input of MUX_2.

Using this approach, two batches can share the same BRAM and a smaller multiplexer can be associated with the second one, resulting in lower resource utilization. The output of the BRAM consists of the 18-bit word shown in Fig. 11. The number of sample points in the two batches cannot exceed the number of bits dedicated for the register enable portion of the memory word, which is 8 bits in our implementation.

After the 37 columns have been output by the 37×1 sliding window, all of the sample points in all of the batches would have received

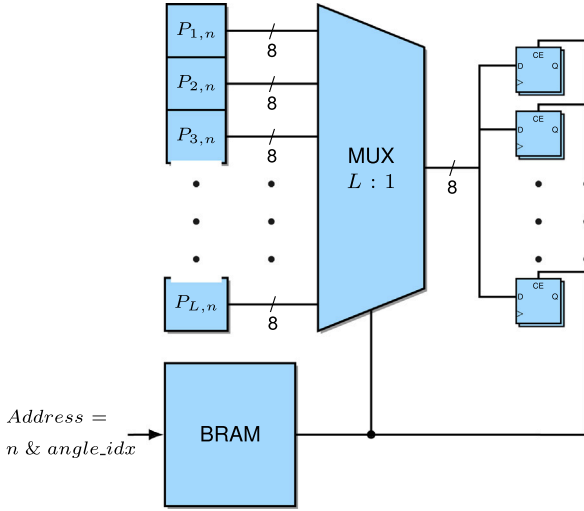


Fig. 9. Dividing the sample points into batches.

their values, and they are passed on to 256 parallel comparators that are responsible for computing the descriptor bits.

It is important to note that the 37×1 window will output the sampling patch for each keypoint that has been detected, and a subsequent keypoint cannot occur while the window is outputting the patch for the previous one. Therefore, a minimal distance has to be ensured between two consecutive keypoints which is accomplished by the NMS component in the Harris module.

3.3. Matcher module

The matching of binary features is conventionally done using a linear search where each descriptor from the second image is compared with all of the descriptors from the first image, and the one closest to it according to the Hamming distance metric is identified as a match. Popular computer vision libraries, such as OpenCV and scikit-image [32], require an additional constraint where two descriptors i and j from the first and second images respectively are considered a match only if i is the best match of j and vice-versa. This is referred to as a *cross-check*, which has not been adopted in previous FPGA implementations of the feature matching process [33–35]. Algorithm 1 details our adaptation of the matching routine, which has been implemented in the PL of the Zynq SoC.

The inputs consist of the two sets of descriptors $Descs1$ and $Descs2$ from the two images each stored in a dual-port RAM. $matches1_2$ is a vector that associates each descriptor from $Descs1$ with its best match from $Descs2$, the distance between the two is stored in $distances1_2$. $matches2_1$ associates each descriptor from $Descs2$ with its best match from $Descs1$. For instance, $matches1_2[i] = j$ means that the j^{th} descriptor from $Descs2$ is the best match of the i^{th} descriptor from $Descs1$.

Fig. 12 depicts the essential elements of the hardware structure for constructing the vectors $matches1_2$, $matches2_1$, and $distances1_2$. A descriptor D_j is read from $Descs2$, then every clock cycle a descriptor D_i is read from $Descs1$ and the Hamming distance is computed between the two using parallel XOR ports, which compute $D_j \oplus D_i$ in a single clock cycle, then a pipelined adder tree is used to calculate the number of bits equal to one in the output of the XOR component. The output of the adder tree is the Hamming distance $Hamming(D_i, D_j)$. Using this pipelined structure a new distance can be computed every clock cycle.

The output of the comparators is asserted when input A is strictly less than input B , and it is equal to zero otherwise. $image_idx$ is a signal that indicates whether the accelerator is processing the first image or

Algorithm 1: Feature matching routine

Input: $Descs1$, $Descs2$

Data: $matches1_2$, $matches2_1$, $distances1_2$

Output: $Matches$

for $j \leftarrow 1$ **to** $size(Descs2)$ **do**

$D_j \leftarrow Descs2[j]$;

$min_dist_j2i \leftarrow \text{any value} > 256$;

for $i \leftarrow 1$ **to** $size(Descs1)$ **do**

$D_i \leftarrow Descs1[i]$;

$dist_j2i \leftarrow Hamming(D_i, D_j)$;

if $dist_j2i < min_dist_j2i$ **then**

$min_dist_j2i \leftarrow dist_j2i$;

$matches2_1[j] \leftarrow i$;

if $dist_j2i < distances1_2[i]$ **then**

$distances1_2[i] \leftarrow dist_j2i$;

$matches1_2[i] \leftarrow j$;

for $k \leftarrow 1$ **to** $\min(size(Descs1), size(Descs2))$ **do**

if $matches2_1[matches1_2[k]] = k$ **and**

$distances1_2[k] < Hamming_threshold$ **then**

$Matches.add([k, matches1_2[k]]);$

the second image in the pair of images whose descriptors are to be matched. A value of zero indicates that the first image is being processed and the descriptors that are output by the descriptor generator are stored in $Descs1$, otherwise the second image is being processed and the descriptors are stored in $Descs2$. The address generators produces the indices of the two nested loops in Algorithm 1.

min_dist_j2i is used to store the distance between a descriptor D_j and its best match from $Descs1$, if the distance between D_i and D_j is less than min_dist_j2i , then i is the new best match to j and min_dist_j2i is updated to store $Hamming(D_i, D_j)$. Since the Hamming distance between two descriptors ranges between 0 and 256, min_dist_j2i and all entries of $distances1_2$ must be initialized with a value greater than 256.

The final loop in Algorithm 1 establishes the matches between the two descriptor sets. Using the cross-check method, two descriptors with indices k and l from images 1 and 2 respectively are matches if $matches1_2[k] = l$ and $matches2_1[l] = k$. Consequently, this implies that $matches2_1[matches1_2[k]] = k$. In addition to satisfying the cross-check condition, the current implementation adds the restriction that two descriptors will be matched only if the distance between them is less than the predefined threshold $Hamming_threshold$.

4. Results and discussions

The proposed architecture was written in VHDL using the Vivado 2019.1 Design Suite and tested on a Zedboard development board housing the Xilinx XC7Z020CLG484-1 SoC. The performance of the proposed architecture is compared with the results obtained using OpenCV 4.5.4 running on a computer with an Intel i5-1035G1 1 GHz processor, 8 GB of RAM, NVIDIA GeForce MX230 GPU, and Windows 10 as the operating system.

In order to conduct a fair comparison between the two implementations, the number of scale pyramid levels in OpenCV was limited to one and the matches with a distance greater than the threshold $Hamming_threshold$ that was used in the hardware implementation are discarded. Moreover, the maximum number of keypoints to be extracted from the images was limited to 500 in both implementations.

The test cases shown in Fig. 13 consist of image pairs taken from the affine covariant features dataset [36]. The first pair of images (*boat* pair) exhibits rotational and scale differences (Fig. 13(a)). The second pair of images (Fig. 13(b)) is composed of the *boat* pair but with a

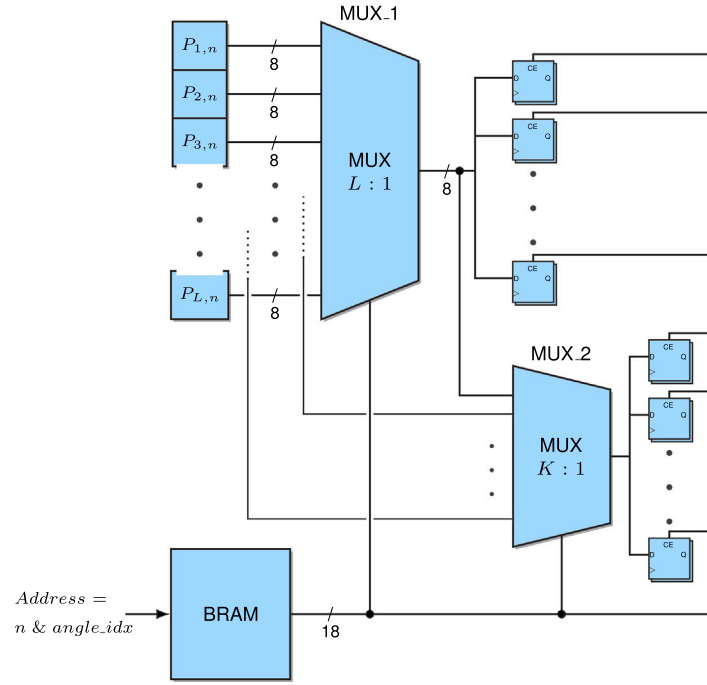


Fig. 10. Combining the batches into pairs.

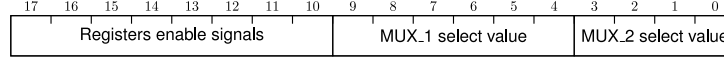
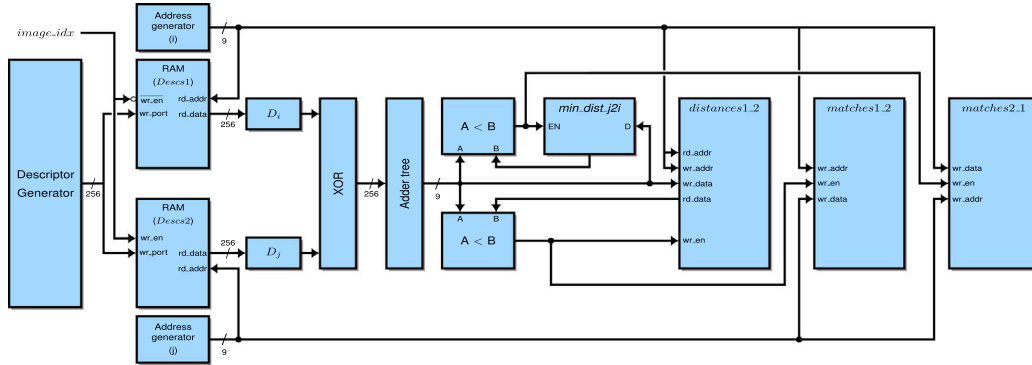


Fig. 11. The format of the BRAM output.

Fig. 12. Hardware architecture for constructing *matches1_2*, *matches2_1*, and *distances1_2*.

synthetically added Gaussian noise with a standard deviation $\sigma = 50$. The *leuven* pair (Fig. 13(c)) comprises images with significant changes in lighting. The *bikes* pair (Fig. 13(d)) depicts images under varying blurring conditions.

The results produced by both OpenCV and the proposed hardware accelerator using a Hamming distance threshold of 60 are presented in Fig. 14. The detected keypoints are marked in orange, inlying matches are shown in green while the outliers are shown in red.

The performance of the detector is evaluated using the repeatability measure which represents the detector's ability to extract keypoints regardless of imaging conditions. The repeatability rate $r_i(\epsilon)$ of image I_i is defined as [37]:

$$r_i(\epsilon) = \frac{|R_i(\epsilon)|}{\min(n_1, n_i)} \quad (13)$$

where

$$R_i(\epsilon) = \{(\tilde{x}_1, \tilde{x}_i) \mid \text{dist}(H_{1i}\tilde{x}_1, \tilde{x}_i) < \epsilon\}$$

is the set of point pairs $(\tilde{x}_1, \tilde{x}_i)$ that lie in the common part of images I_1 and I_i , n_1 and n_i are the cardinalities of the two sets of points $\{\tilde{x}_1\}$

Table 1

Comparison between OpenCV and the proposed architecture in terms of repeatability and precision.

		Repeatability ($\epsilon = 1.5$)	Precision	Latency (ms)
Boat	OpenCV	0.576	0.976	18.96
	Proposed	0.522	0.992	6.12
Boat + noise	OpenCV	0.356	0.811	18.9
	Proposed	0.369	0.958	6.12
Leuven	OpenCV	0.421	0.906	12.94
	Proposed	0.509	0.938	5.76
Bikes	OpenCV	0.25	0.784	14.34
	Proposed	0.244	0.824	7.39

and $\{\tilde{x}_i\}$, respectively. H_{1i} is the homography relating the points from image I_1 to the ones in image I_i .

The descriptor is evaluated according to the precision metric [38]. A value close to one would indicate that the predominance of matches

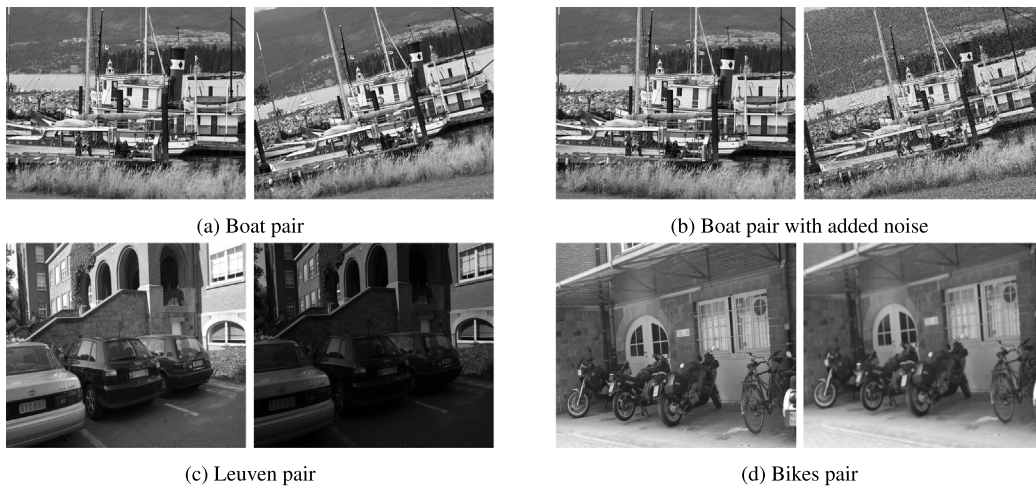


Fig. 13. The test images used to evaluate the performance of the proposed architecture.

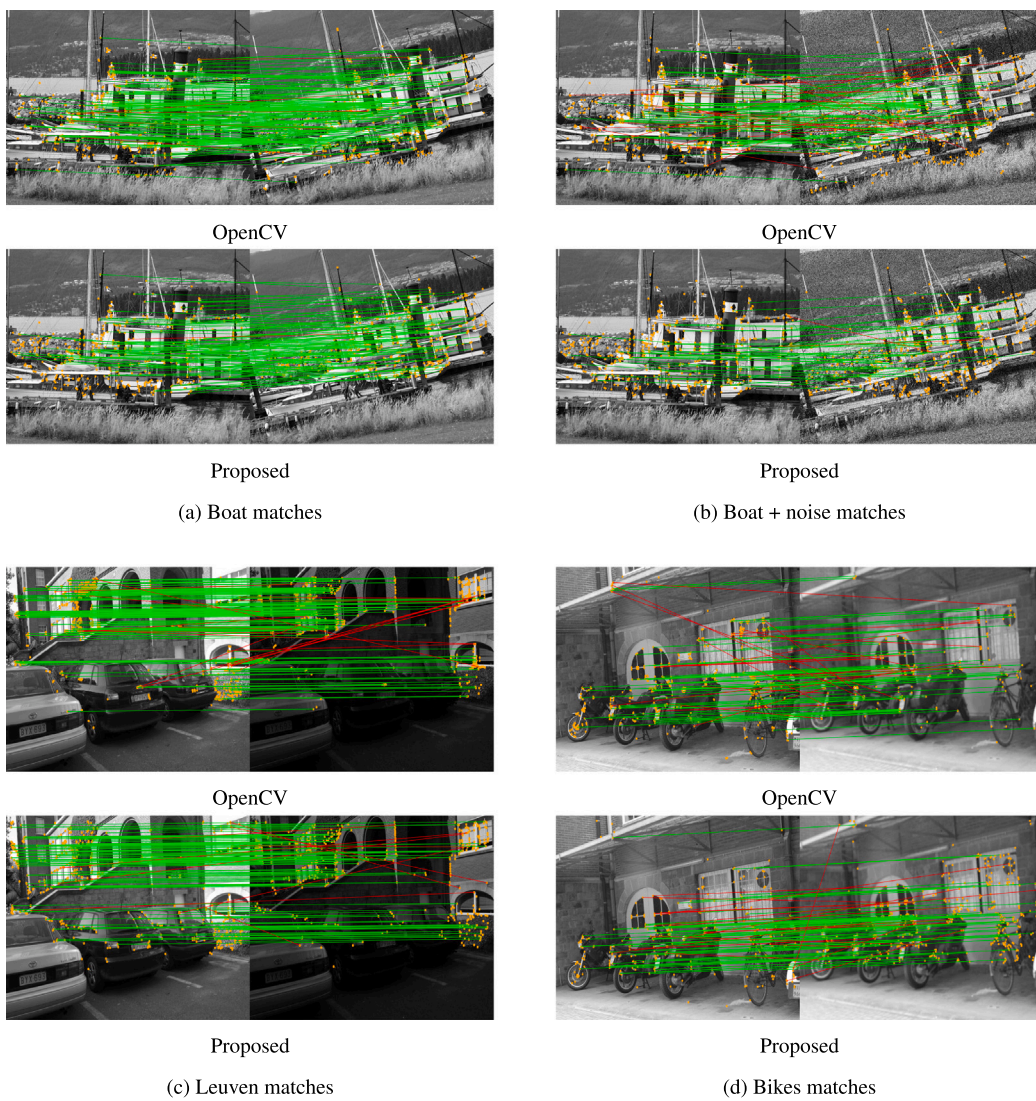


Fig. 14. The matches found by both OpenCV and the proposed hardware on the four test cases.

Table 2
Performance comparison with previous ORB implementations.

Implementation	Image resolution	Clock freq. (MHz)	Latency (ms)	Frame rate (FPS)	Throughput (MPix/s)	Clock cycles per pixel
[13]	640 × 480	100	9	111.11	34	3
[25]	1920 × 1080	150	13.8	72.3	150	1
[15]	640 × 480	203	14.8	67	21	10
[26]	1920 × 1080	175	11.9	84.4	175	1
[27]	1920 × 1080	184	11.2	88.8	184	1
Proposed	850 × 680	100	6.12	163.4	94	1

Table 3
Resource utilization of the proposed design.

	LUTs	Registers	DSP	Memory (Kb)
Harris	2744 (5.16%)	3363 (3.16%)	3 (1.36%)	324 (6.43%)
rBRIEF	10,203 (19.18%)	8227 (7.73%)	12 (5.45%)	2862 (56.79%)
Matcher	1012 (1.9%)	709 (0.67%)	0 (0%)	288 (5.71%)
Total	14,535 (27.32%)	12,828 (12.06%)	15 (6.82%)	3582 (71.07%)

returned by the algorithm are inliers, making them ideal for subsequent processing using robust estimators. The precision is defined as:

$$\text{precision} = \frac{\# \text{ correct matches}}{\# \text{ correct matches} + \# \text{ false matches}} \quad (14)$$

the correct matches are those with an error of 5 pixels or less.

Table 1 provides a summary of the results achieved by OpenCV and the proposed hardware architecture. As can be seen, the two implementations yield very similar results for all four test cases. In terms of repeatability, the reference implementation marginally outperformed the proposed architecture in the case of *boat* and *bikes* images. The similar results suggest that the implemented Harris detector can serve as a good replacement for the reference detector that combines FAST and Harris. The proposed architecture, however, achieved better precision on all test cases, even in the presence of noise.

The latency required for extracting the keypoints and descriptors from a single image in the *boat* pair with a resolution of 850 × 680 was 6.12 ms using a clock with a frequency of 100 MHz, which is equivalent to a throughput of 94 MPix/s or a frame rate of 163 fps. This is consistent with the throughput achieved using an image from the *leuven* pair with a resolution of 900 × 600 and using an image from *bikes* pair with a resolution of 1000 × 700 pixels. The latency reported in the case of OpenCV is the average of 100 runs.

The similar performance in terms of repeatability and precision between the proposed architecture and OpenCV is achieved without impacting the throughput of the implementation, as can be seen in the comparison with previous works in Table 2. Since the various implementations use different clock frequencies, the comparison is based on the number of clock cycles required to process a single pixel, which is obtained by [13]:

$$\text{clock cycles per pixel} = \frac{\text{frequency} \times \text{latency}}{\text{image resolution}} \quad (15)$$

The proposed architecture achieves an acceptable processing rate compared to the stream-oriented architectures [25–27], and a higher throughput compared to the non-streaming architectures [13,15] while preserving the quality and robustness of the reference ORB algorithm, namely its invariance to rotation and noise.

The resources required to map the proposed architecture into hardware are listed in Table 3. Overall, the design requires a relatively small footprint but relies heavily on block RAMs, especially in the descriptor generation unit of the rBRIEF module, to implement the look-up tables associated with the sample point batches. The total utilization refers to the resources required to implement the entire architecture including the control module and the pre-filtering.

It is worth highlighting that using a single unfiltered image patch for both the moments and the descriptor extraction, as is the case in [23,25,26] would negate the need to add the 31 × 1 sliding window, and would lead to lower logic and BRAM utilization. But this would come at the detriment of the descriptor's noise immunity.

5. Conclusion

This paper introduced a new hardware architecture aimed at accelerating the feature detection and matching task using the ORB descriptor. The work was motivated by the need for more faithful adaptations of the ORB algorithm targeting FPGA platforms. Included in this paper are methods for the accurate and efficient computation of the patch moments, generation of the descriptor bits in a single clock cycle without rotating the sample point coordinates, and the matching of the descriptors in hardware. A throughput of 94 MPix/s was attained while requiring minimal hardware resources. The proposed accelerator exhibited a similar performance in terms of repeatability and precision compared to the reference implementation in OpenCV when tested on images with several photometric and geometric variations.

Future work will focus on incorporating the proposed design within a fully automated system for the analysis and interpretation of remote sensing imagery where the high throughput and robustness of the implementation are indispensable.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The research being reported in this publication was supported by the Algerian Directorate General for Scientific Research and Technological Development (DGRSDT), Algeria.

References

- [1] D.G. Lowe, Distinctive image features from scale-invariant keypoints, *Int. J. Comput. Vis.* 60 (2) (2004) 91–110, <http://dx.doi.org/10.1023/B:VISI.0000029664.99615.94>.
- [2] H. Bay, A. Ess, T. Tuytelaars, L. Van Gool, Speeded-up robust features (SURF), *Comput. Vis. Image Underst.* 110 (3) (2008) 346–359, <http://dx.doi.org/10.1016/j.cviu.2007.09.014>, Similarity Matching in Computer Vision and Multimedia.
- [3] M. Calonder, V. Lepetit, C. Strecha, P. Fua, BRIEF: Binary robust independent elementary features, 6314, 2010, pp. 778–792, http://dx.doi.org/10.1007/978-3-642-15561-1_56.
- [4] E. Rublee, V. Rabaud, K. Konolige, G. Bradski, ORB: An efficient alternative to SIFT or SURF, in: 2011 International Conference on Computer Vision, 2011, pp. 2564–2571, <http://dx.doi.org/10.1109/ICCV.2011.6126544>.

- [5] E. Karami, S. Prasad, M. Shehata, Image matching using SIFT, SURF, BRIEF and ORB: performance comparison for distorted images, 2017, ArXiv preprint [arXiv:1710.02726](https://arxiv.org/abs/1710.02726).
- [6] J. Figat, T. Kornuta, W. Kasprzak, Performance evaluation of binary descriptors of local features, in: L.J. Chmielewski, R. Kozera, B. Shin, K.W. Wojciechowski (Eds.), Computer Vision and Graphics - International Conference, ICCVG 2014, Warsaw, Poland, September 15-17, 2014. Proceedings, in: Lecture Notes in Computer Science, vol. 8671, Springer, 2014, pp. 187–194, [http://dx.doi.org/10.1007/978-3-319-11331-9_23](https://doi.org/10.1007/978-3-319-11331-9_23).
- [7] c. İşı k, A comparative evaluation of well-known feature detectors and descriptors, Int. J. Appl. Math. Electron. Comput. 3 (2014) [http://dx.doi.org/10.18100/ijamec.60004](https://doi.org/10.18100/ijamec.60004).
- [8] P. Kreowsky, B. Stabernack, A full-featured FPGA-based pipelined architecture for SIFT extraction, IEEE Access 9 (2021) 128564–128573, [http://dx.doi.org/10.1109/ACCESS.2021.3104387](https://doi.org/10.1109/ACCESS.2021.3104387).
- [9] T. Sledevič, A. Serackis, SURF algorithm implementation on FPGA, in: 2012 13th Biennial Baltic Electronics Conference, 2012, pp. 291–294, [http://dx.doi.org/10.1109/BEC.2012.6376874](https://doi.org/10.1109/BEC.2012.6376874).
- [10] G. Bradski, The opencv library, Dr. Dobb's J. Softw. Tools (2000).
- [11] M.A. Alnagdawi, S.Z. Mohd Hashim, ORB-PC feature-based image registration, in: 2019 IEEE International Conference on Signal and Image Processing Applications, ICSIPA, 2019, pp. 111–115, [http://dx.doi.org/10.1109/ICSIPA45851.2019.8977793](https://doi.org/10.1109/ICSIPA45851.2019.8977793).
- [12] J. Chen, L. Luo, S. Wang, H. Wu, Automatic panoramic UAV image mosaic using ORB features and robust transformation estimation, in: 2018 37th Chinese Control Conference, CCC, 2018, pp. 4265–4270, [http://dx.doi.org/10.23919/ChiCC.2018.8483513](https://doi.org/10.23919/ChiCC.2018.8483513).
- [13] K.-y. Lee, A design of an optimized ORB accelerator for real-time feature detection, Int. J. Control Autom. 7 (2014) 213–218, [http://dx.doi.org/10.14257/ijca.2014.7.3.20](https://doi.org/10.14257/ijca.2014.7.3.20).
- [14] E. Rosten, T. Drummond, Machine learning for high-speed corner detection, in: A. Leonardis, H. Bischof, A. Pinz (Eds.), Computer Vision – ECCV 2006, Springer Berlin Heidelberg, Berlin, Heidelberg, 2006, pp. 430–443.
- [15] W. Fang, Y. Zhang, B. Yu, S. Liu, FPGA-based ORB feature extraction for real-time visual SLAM, in: 2017 International Conference on Field Programmable Technology, ICFPT, 2017, pp. 275–278, [http://dx.doi.org/10.1109/FPT.2017.8280159](https://doi.org/10.1109/FPT.2017.8280159).
- [16] R. Sun, P. Liu, J. Wang, Z. Zhou, A low latency feature extraction accelerator with reduced internal memory, in: 2017 IEEE International Symposium on Circuits and Systems, ISCAS, 2017, pp. 1–4, [http://dx.doi.org/10.1109/ISCAS.2017.8050258](https://doi.org/10.1109/ISCAS.2017.8050258).
- [17] W. Zhu, L. Liu, G. Jiang, S. Yin, S. Wei, A 135 fps 1080p 87.5 mW binary descriptor based image feature extraction accelerator, IEEE Trans. Circuits Syst. Video Technol. 26 (2015) 1, [http://dx.doi.org/10.1109/TCSVT.2015.2469116](https://doi.org/10.1109/TCSVT.2015.2469116).
- [18] R. Sun, J. Qian, R.H. Jose, Z. Gong, R. Miao, W. Xue, P. Liu, A flexible and efficient real-time ORB-based full-HD image feature extraction accelerator, IEEE Trans. Large Scale Integr. (VLSI) Syst. 28 (2) (2020) 565–575, [http://dx.doi.org/10.1109/TVLSI.2019.2945982](https://doi.org/10.1109/TVLSI.2019.2945982).
- [19] R. Sun, P. Liu, J. Wang, C. Accetti, A.A. Naqvi, A 42fps full-HD ORB feature extraction accelerator with reduced memory overhead, in: 2017 International Conference on Field Programmable Technology, ICFPT, 2017, pp. 183–190, [http://dx.doi.org/10.1109/FPT.2017.8280137](https://doi.org/10.1109/FPT.2017.8280137).
- [20] R. de Lima, J. Martinez-Carranza, A. Morales-Reyes, R. Cumplido, Improving the construction of ORB through FPGA-based acceleration, Mach. Vis. Appl. 28 (2017) [http://dx.doi.org/10.1007/s00138-017-0851-5](https://doi.org/10.1007/s00138-017-0851-5).
- [21] R. Liu, J. Yang, Y. Chen, W. Zhao, eSLAM: An energy-efficient accelerator for real-time ORB-SLAM on FPGA platform, in: Proceedings of the 56th Annual Design Automation Conference 2019, DAC'19, Association for Computing Machinery, New York, NY, USA, 2019, [http://dx.doi.org/10.1145/3316781.3317820](https://doi.org/10.1145/3316781.3317820).
- [22] T. Imsaengsuk, S. Pumrin, Feature detection and description based on ORB algorithm for FPGA-based image processing, in: 2021 9th International Electrical Engineering Congress, IEECON, 2021, pp. 420–423, [http://dx.doi.org/10.1109/IEECON51072.2021.9440232](https://doi.org/10.1109/IEECON51072.2021.9440232).
- [23] J. Weberruss, L. Kleeman, T. Drummond, ORB feature extraction and matching in hardware, in: R. Mahony, J. Kim, H. Li (Eds.), Australasian Conference on Robotics and Automation, ACRA 2015, Australian Robotics and Automation Association (ARAA), Australia, 2015, pp. 61–70, Australasian Conference on Robotics and Automation 2015, ACRA 2015 ; Conference date: 02-12-2015 Through 04-12-2015, URL <http://www.araa.asn.au/conferences/acra-2015/>.
- [24] C. Harris, M. Stephens, A combined corner and edge detector, in: Proceedings of the 4th Alvey Vision Conference, 1988, pp. 147–151.
- [25] J. Weberruss, L. Kleeman, D. Boland, T. Drummond, FPGA acceleration of multi-level ORB feature extraction for computer vision, in: 2017 27th International Conference on Field Programmable Logic and Applications, FPL, IEEE, 2017, pp. 1–8, [http://dx.doi.org/10.23919/FPL.2017.8056856](https://doi.org/10.23919/FPL.2017.8056856).
- [26] T.H. Pham, P. Tran, S.-K. Lam, High-throughput and area-optimized architecture for rBRIEF feature extraction, IEEE Transactions on Very Large Scale Integration (VLSI) Systems 27 (4) (2019) 747–756, [http://dx.doi.org/10.1109/TVLSI.2018.2881105](https://doi.org/10.1109/TVLSI.2018.2881105).
- [27] H. Chen, F. Li, Z. Zhang, A bucket-stream rBRIEF extraction architecture for SLAM applications on embedded platforms, in: 2020 International Conference on Field-Programmable Technology, ICFPT, 2020, pp. 277–280, [http://dx.doi.org/10.1109/ICFPT51103.2020.00047](https://doi.org/10.1109/ICFPT51103.2020.00047).
- [28] A. Amarica, C.-E. Gavrilu, O. Boncalo, An FPGA sliding window-based architecture harris corner detector, in: 2014 24th International Conference on Field Programmable Logic and Applications, FPL, 2014, pp. 1–4, [http://dx.doi.org/10.1109/FPL.2014.6927402](https://doi.org/10.1109/FPL.2014.6927402).
- [29] T.L. Chao, K.H. Wong, An efficient FPGA implementation of the harris corner feature detector, in: 2015 14th IAPR International Conference on Machine Vision Applications, MVA, 2015, pp. 89–93, [http://dx.doi.org/10.1109/MVA.2015.7153140](https://doi.org/10.1109/MVA.2015.7153140).
- [30] J. Sánchez, N. Monzón, A. Salgado, An analysis and implementation of the harris corner detector, Image Process. Line 8 (2018) 305–328, [http://dx.doi.org/10.5201/ipol.2018.229](https://doi.org/10.5201/ipol.2018.229).
- [31] S. Khorbotly, F. Hassan, A modified approximation of 2D Gaussian smoothing filters for fixed-point platforms, in: 2011 IEEE 43rd Southeastern Symposium on System Theory, 2011, pp. 151–159, [http://dx.doi.org/10.1109/SSST.2011.5753797](https://doi.org/10.1109/SSST.2011.5753797).
- [32] S. van der Walt, J.L. Schönberger, J. Nunez-Iglesias, F. Boulogne, J.D. Warner, N. Yager, E. Goullart, T. Yu, the scikit-image contributors, Scikit-image: image processing in python, PeerJ 2 (2014) e453, [http://dx.doi.org/10.7717/peerj.453](https://doi.org/10.7717/peerj.453).
- [33] H. Heo, J.-y. Lee, K.-y. Lee, C.-h. Lee, FPGA based implementation of FAST and BRIEF algorithm for object recognition, in: 2013 IEEE International Conference of IEEE Region 10, TENCON 2013, 2013, pp. 1–4, [http://dx.doi.org/10.1109/TENCON.2013.6718842](https://doi.org/10.1109/TENCON.2013.6718842).
- [34] M. Fularz, M. Kraft, A. Schmidt, A. Kasiński, A high-performance FPGA-based image feature detector and matcher based on the FAST and BRIEF algorithms, Int. J. Adv. Robot. Syst. 12 (10) (2015) 141, [http://dx.doi.org/10.5772/61434](https://doi.org/10.5772/61434).
- [35] J. Huang, G. Zhou, X. Zhou, R. Zhang, A new FPGA architecture of FAST and BRIEF algorithm for on-board corner detection and matching, Sensors 18 (4) (2018) [http://dx.doi.org/10.3390/s18041014](https://doi.org/10.3390/s18041014), URL <https://www.mdpi.com/1424-8220/18/4/1014>.
- [36] K. Mikolajczyk, T. Tuytelaars, C. Schmid, A. Zisserman, J. Matas, F. Schaffalitzky, T. Kadir, L. Van Gool, A comparison of affine region detectors, Int. J. Comput. Vis. 65 (2005) 43–72, [http://dx.doi.org/10.1007/s11263-005-3848-x](https://doi.org/10.1007/s11263-005-3848-x).
- [37] C. Schmid, R. Mohr, C. Bauckhage, Evaluation of interest point detectors, Int. J. Comput. Vis. 37 (2000) 151–172, [http://dx.doi.org/10.1023/A:1008199403446](https://doi.org/10.1023/A:1008199403446).
- [38] K. Mikolajczyk, C. Schmid, A performance evaluation of local descriptors, IEEE Trans. Pattern Anal. Mach. Intell. 27 (10) (2005) 1615–1630, [http://dx.doi.org/10.1109/TPAMI.2005.188](https://doi.org/10.1109/TPAMI.2005.188).



Nadir Mohamed Belmessaoud received his Master's degree in embedded electronics from Ibn Khaldoun university of Tiaret, Algeria in 2019, and is currently a Ph.D. student associated with the Communication Networks, Architectures and Multimedia (RCAM) laboratory in Djillali Liabès university of Sidi Bel Abbès. His principal research interests are in the fields of reconfigurable hardware, remote sensing and image processing.



Youcef Bentoutou received his Engineer, Magister, and Doctorate degrees in Electrical Engineering from the University of Sidi Bel Abbès (Algeria) in 1997, 2000, and 2004 respectively. He is currently a Research Director of system architecture and signal processing at the Satellite Development Center (CDS: Centre de développement des Satellites) of the Algerian Space Agency. He is also an associate researcher at the 'Communication Networks, Architectures and Multimedia (RCAM)' Laboratory of the University of Sidi Bel Abbès. Dr. Bentoutou received the 2009 TWAS-AAS-Microsoft award for young scientists for his contributions to the fields of pattern recognition and information processing. His principal research interests are in the fields of remote sensing, satellite onboard data handling, and space radiation environment and effects analysis and mitigation.



Miloud Chikr El-Mezouar is currently a professor in the Department of Electronic, Electrical Engineering Faculty, Djillali Liabes University of Sidi Bel Abbas, in Algeria, where he has been teaching since 2003. He received the Dipl.Ing. and Magister degrees in electrical engineering from the University of Djillali Liabes, Sidi Bel Abbas, Algeria, in 1996 and 1999, respectively. Then he obtained the Ph.D. degree in signal and image processing, under joint supervision, from the University of Djillali Liabes and from the Institut National des Sciences Appliquées (INSA), Rennes, France in 2012. In December 2014, Miloud Chikr El-Mezouar has gotten the university habilitation to direct research in

signal and image processing domain, from Djillali Liabes University of Sidi Bel Abbas, Algeria. As a part of the “Communication Networks, Architecture, and Multimedia” Laboratory – He is now supervising 6 Ph.D. students. – His principal research interests are in the fields of digital signal and image processing, Remote sensing, image analysis, medical and satellite image applications, pattern recognition, electrical and computer engineering, Embedded systems and artificial olfaction systems. He participates in the national project (2016–2019) on the implementation of electronic noses for medical diagnosis.